

TRABAJO PRÁCTICO N° 1

Seminario de Práctica de Informática

Sistema de Gestión Integral para una Clínica de Salud

Alumno: Antonella Diaz

DNI: 28.910.424

Comisión: VINFOR12606

Índice

1. Introducción
2. Justificación
3. Objetivo General
4. Objetivos Específicos
5. Objetivo del Sistema
6. Definición del Proyecto
7. Elicitación de Requerimientos
8. Conocimiento del Negocio
9. Propuesta de Solución
10. Actores del Sistema
11. Casos de Uso
12. Diagramas de Asociación
13. Requerimientos Funcionales
14. Requerimientos No Funcionales

1. Introducción

En la actualidad, las instituciones de salud enfrentan un desafío creciente en la gestión eficiente de sus operaciones internas. La administración de turnos, historias clínicas, inventarios y facturación suele realizarse de manera manual o mediante sistemas desconectados entre sí, lo que genera duplicación de información, demoras en la atención y una mayor probabilidad de cometer errores.

Ante esta problemática, el presente trabajo propone el desarrollo de un Sistema de Gestión Integral para una Clínica de Salud. El sistema busca centralizar la información y automatizar los procesos administrativos y médicos más relevantes, permitiendo que el personal de la clínica — médicos, recepcionistas y administrativos— pueda acceder a los datos de forma rápida, segura y ordenada.

El proyecto se desarrollará utilizando el lenguaje Java con persistencia de datos en una base de datos MySQL, siguiendo los principios del Proceso Unificado de Desarrollo (PUD) y aplicando el patrón de diseño DAO para el acceso a los datos.

A lo largo del documento se presentan los fundamentos que justifican el proyecto, la definición del sistema, los requerimientos identificados y los casos de uso que describen la interacción entre los usuarios y el sistema.

2. Justificación

Por qué se hace: La clínica de salud presenta actualmente una gestión fragmentada de sus procesos internos. La falta de integración entre las distintas áreas —recepción, consultorios, farmacia y administración— genera demoras, errores en el registro de información y dificultades para obtener datos en tiempo real. Esta situación impacta negativamente tanto en la calidad del servicio como en la experiencia del paciente.

Para qué se hace: El desarrollo de un Sistema de Gestión Integral permitirá unificar todos los procesos en una única plataforma digital, reduciendo los errores operativos, optimizando los tiempos de atención y mejorando la toma de decisiones mediante el acceso centralizado a la información. De esta manera, tanto el personal como los pacientes se verán beneficiados por un servicio más eficiente, organizado y confiable.

3. Objetivo General

Desarrollar un Sistema de Gestión Integral para una Clínica de Salud que centralice la administración de pacientes, médicos, turnos, historias clínicas, inventarios y facturación en una única plataforma, mejorando la eficiencia operativa y la calidad del servicio brindado.

4. Objetivos Específicos

Los siguientes objetivos específicos representan los logros concretos que se alcanzarán en el marco del objetivo general planteado:

- Lograr el registro y la gestión centralizada de la información de pacientes y médicos en una base de datos relacional.
- Lograr la programación, modificación y cancelación de turnos médicos de forma digital y ordenada.
- Lograr el registro y consulta de historias clínicas electrónicas vinculadas a cada paciente y médico.
- Lograr el control del inventario de medicamentos e insumos con seguimiento de stock y fechas de vencimiento.
- Lograr la generación y seguimiento de facturas asociadas a los servicios prestados a los pacientes.
- Lograr la generación de reportes estadísticos que faciliten la toma de decisiones administrativas.

5. Objetivo del Sistema

El sistema tiene como finalidad gestionar de manera integral las operaciones de la clínica de salud, cubriendo todos los procesos desde el registro de un nuevo paciente hasta la generación de la factura por los servicios recibidos.

Límites del sistema:

Desde: el ingreso de un nuevo paciente al sistema mediante su registro con datos personales.

Hasta: la emisión de la factura correspondiente a los servicios médicos brindados y el registro del pago en el sistema.

El sistema no contempla la integración con obras sociales externas ni con sistemas de laboratorio de terceros, quedando fuera del alcance todo procesamiento que ocurra fuera de la plataforma.

6. Definición del Proyecto

El proyecto consiste en el desarrollo e implementación de un sistema informático que permita gestionar de manera integral las operaciones de una clínica de salud. El sistema integrará módulos interconectados de gestión de pacientes, médicos, turnos, historias clínicas, inventarios, facturación y reportes, todos vinculados a una base de datos centralizada.

El sistema será escalable y adaptable a distintos tamaños de clínicas, diseñado para optimizar el funcionamiento general de la organización y reducir la dependencia de procesos manuales.

7. Elicitación de Requerimientos

Para identificar las necesidades del sistema se llevó a cabo un proceso de elicitación que permitió relevar la información necesaria para definir con precisión los requerimientos. Las técnicas utilizadas incluyeron entrevistas con médicos, recepcionistas y personal administrativo de la clínica, la observación directa de los procesos actuales, el análisis de la documentación existente y la identificación de los principales problemas y necesidades de cada área.

Este proceso permitió definir de manera clara y completa los requerimientos funcionales y no funcionales del sistema, constituyendo la base para el diseño y desarrollo del mismo.

8. Conocimiento del Negocio

La clínica de salud opera a través de cuatro procesos clave que el sistema deberá contemplar. En primer lugar, la atención de pacientes comprende la asignación de turnos y la realización de consultas médicas. En segundo lugar, el registro de información médica abarca la confección y actualización de las historias clínicas digitales. En tercer lugar, la gestión de medicamentos e insumos implica el control del stock y el seguimiento de fechas de vencimiento. Por último, la facturación y el cobro de servicios requieren la emisión de comprobantes y el registro de los pagos realizados.

Actualmente estos procesos se llevan a cabo de forma manual o con herramientas poco integradas, lo que genera demoras, errores y ausencia de información en tiempo real. El sistema propuesto permitirá digitalizar y unificar estos procesos, mejorando la eficiencia operativa y la calidad del servicio.

9. Propuesta de Solución

Se propone el desarrollo de un sistema informático centralizado que integre todas las áreas de la clínica mediante una única plataforma. El sistema contará con una interfaz de consola desarrollada en Java, conectada a una base de datos MySQL mediante el driver JDBC. El acceso estará organizado por roles —administrador, médico y recepcionista— garantizando que cada usuario acceda únicamente a las funcionalidades correspondientes a su perfil.

La arquitectura del sistema seguirá el patrón de diseño DAO (Data Access Object), que permite separar la lógica de acceso a los datos de la lógica de negocio, favoreciendo el mantenimiento y la escalabilidad del sistema. Esto permitirá mejorar la gestión, reducir errores y optimizar los tiempos de atención.

10. Actores del Sistema

Se identificaron los siguientes actores que interactúan con el sistema:

Recepcionista: Es el actor principal del sistema. Se encarga de registrar nuevos pacientes, programar, modificar y cancelar turnos, y emitir facturas por los servicios prestados.

Médico: Accede a las historias clínicas de sus pacientes, registra diagnósticos y tratamientos, y consulta su agenda de turnos asignados.

Administrador: Gestiona el inventario de medicamentos e insumos, genera reportes estadísticos y administra los usuarios del sistema.

Paciente: Actor externo que solicita turnos y recibe atención médica. No interactúa directamente con el sistema, sino a través de la recepcionista.

11. Casos de Uso

Se identificaron los siguientes casos de uso principales que describen la interacción entre los actores y el sistema:

- Registrar paciente
- Programar turno
- Modificar turno
- Cancelar turno
- Registrar consulta médica / Historia clínica
- Consultar historia clínica
- Gestionar inventario
- Emitir factura
- Generar reporte

Fichas de Casos de Uso:

Nombre del Caso de Uso	Registrar Paciente
Actores	Recepcionista
Descripción	El sistema permite registrar un nuevo paciente ingresando sus datos personales.
Precondición	El paciente no debe estar registrado previamente en el sistema.
Postcondición	El paciente queda registrado en la base de datos y disponible para asignarle turnos.
Flujo Principal	1. La recepcionista selecciona la opción 'Gestión de Pacientes'. 2. Selecciona 'Agregar paciente'. 3. Ingresa nombre, apellido, fecha de nacimiento, dirección, teléfono y correo. 4. El sistema guarda los datos en la base de datos. 5. El sistema confirma el registro exitoso.

Flujo Alternativo	Si el paciente ya existe, el sistema muestra un mensaje de error y no permite el duplicado.
--------------------------	---

Nombre del Caso de Uso	Programar Turno
Actores	Recepcionista
Descripción	El sistema permite asignar un turno médico a un paciente registrado.
Precondición	El paciente y el médico deben estar registrados en el sistema.
Postcondición	El turno queda registrado con estado 'Pendiente' en la base de datos.
Flujo Principal	1. La recepcionista selecciona 'Gestión de Citas'. 2. Selecciona 'Agregar cita'. 3. El sistema muestra la lista de pacientes y médicos disponibles. 4. La recepcionista selecciona el paciente, el médico, la fecha/hora y el estado. 5. El sistema registra el turno en la base de datos. 6. El sistema confirma la programación exitosa.
Flujo Alternativo	Si no hay pacientes o médicos registrados, el sistema no permite continuar.

Nombre del Caso de Uso	Registrar Historia Clínica
Actores	Médico
Descripción	El sistema permite registrar los detalles de una consulta médica en la historia clínica del paciente.
Precondición	El paciente y el médico deben estar registrados. Debe existir una cita previa.
Postcondición	La historia clínica queda registrada y vinculada al paciente y al médico.
Flujo Principal	1. El médico selecciona 'Historias Clínicas'. 2. Selecciona 'Agregar historia'. 3. El sistema muestra los pacientes y médicos disponibles. 4. El médico selecciona el paciente, ingresa la fecha y los detalles de la consulta. 5. El sistema guarda la historia clínica. 6. El sistema confirma el registro exitoso.

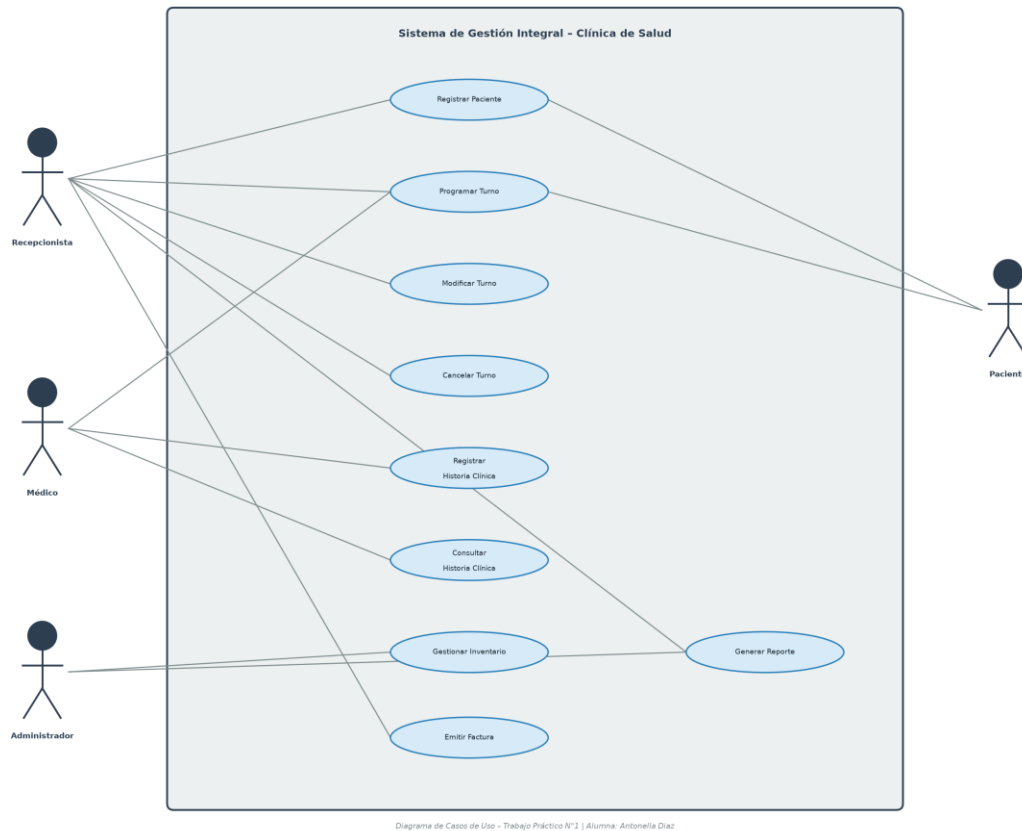
Flujo Alternativo	Si no existen pacientes registrados, el sistema muestra un aviso y cancela la operación.
--------------------------	--

Nombre del Caso de Uso	Emitir Factura
Actores	Recepcionista
Descripción	El sistema permite generar una factura por los servicios médicos prestados a un paciente.
Precondición	El paciente debe estar registrado en el sistema.
Postcondición	La factura queda registrada en la base de datos con su estado (Pendiente o Pagada).
Flujo Principal	1. La recepcionista selecciona 'Facturación y Pagos'. 2. Selecciona 'Agregar factura'. 3. El sistema muestra la lista de pacientes registrados. 4. La recepcionista selecciona el paciente, ingresa fecha, monto total y estado. 5. El sistema registra la factura en la base de datos. 6. El sistema confirma la emisión exitosa.
Flujo Alternativo	Si no hay pacientes registrados, el sistema no permite generar la factura.

Nombre del Caso de Uso	Gestionar Inventario
Actores	Administrador
Descripción	El sistema permite visualizar y agregar productos al inventario de la clínica.
Precondición	El administrador debe estar autenticado en el sistema.
Postcondición	El producto queda registrado en el inventario con su cantidad y fecha de vencimiento.
Flujo Principal	1. El administrador selecciona 'Gestión de Inventarios'. 2. Selecciona 'Agregar producto'. 3. Ingresa nombre del producto, cantidad, fecha de vencimiento e ID de proveedor. 4. El sistema registra el producto en la base de datos. 5. El sistema confirma el alta exitosa.
Flujo Alternativo	Si el proveedor ingresado no existe, el

	sistema registra el producto sin proveedor asociado.
--	--

12. Diagramas de Asociación



13. Requerimientos Funcionales

Los requerimientos funcionales definen las funcionalidades concretas que el sistema debe proveer. A continuación se describen organizados por módulo:

RF01 - Gestión de Pacientes: El sistema debe permitir registrar nuevos pacientes con sus datos personales (nombre, apellido, fecha de nacimiento, dirección, teléfono y correo electrónico), así como consultar el listado completo de pacientes registrados.

RF02 - Gestión de Médicos: El sistema debe permitir registrar médicos indicando nombre, apellido, especialidad, teléfono y correo electrónico, y consultar el listado de médicos disponibles.

RF03 - Gestión de Citas: El sistema debe permitir programar turnos asignando un paciente, un médico, una fecha/hora y un estado (Pendiente, Confirmada o Cancelada). Debe mostrar las citas registradas con los datos completos de paciente y médico.

RF04 - Historias Clínicas: El sistema debe permitir registrar y consultar historias clínicas electrónicas vinculadas a un paciente y un médico, incluyendo la fecha de la consulta y los detalles del diagnóstico o tratamiento.

RF05 - Gestión de Inventarios: El sistema debe permitir registrar productos en el inventario con nombre, cantidad, fecha de vencimiento y proveedor asociado, y mostrar el listado actualizado del stock disponible.

RF06 - Facturación: El sistema debe permitir generar facturas asociadas a un paciente, indicando fecha, monto total y estado de pago (Pendiente o Pagada), y consultar el historial de facturas emitidas.

RF07 - Reportes: El sistema debe generar un reporte de gestión que incluya el total de pacientes, médicos y citas pendientes, el promedio de cantidad de productos en inventario y el total de ingresos por facturas pagadas.

14. Requerimientos No Funcionales

Seguridad: El sistema debe garantizar la protección de los datos de pacientes y médicos. El acceso a las funcionalidades estará restringido según el rol del usuario, asegurando que cada actor solo pueda operar dentro de su área de responsabilidad.

Escalabilidad: El sistema debe estar diseñado de forma modular mediante el patrón DAO, lo que permitirá incorporar nuevos módulos o entidades sin afectar el funcionamiento del sistema existente.

Usabilidad: La interfaz de consola debe ser clara e intuitiva, guiando al usuario con menús numerados y mensajes descriptivos en cada paso, de modo que no se requiera capacitación técnica avanzada para su uso.

Disponibilidad: El sistema debe garantizar el acceso continuo a la información durante el horario de funcionamiento de la clínica. La base de datos MySQL debe contar con copias de seguridad periódicas para evitar pérdida de información.

Rendimiento: El sistema debe responder a las operaciones de consulta e inserción en la base de datos en un tiempo razonable, asegurando una experiencia fluida para el usuario aun cuando existan múltiples registros almacenados.

TRABAJO PRÁCTICO N° 2

Seminario de Práctica de Informática

Sistema de Gestión Integral para una Clínica de Salud

Alumna: Antonella Diaz

DNI: 28.910.424

Comisión: VINFOR12606

Repositorio GitHub: <https://github.com/antonellaanahi/SEMINARIODEPRACTICA>

Índice

1. Etapa de Análisis
2. Etapa de Diseño
3. Diagramas de Secuencia
4. Etapa de Implementación
5. Etapa de Pruebas
6. Definición de Base de Datos
7. Diagrama Entidad-Relación
8. Creación de Tablas MySQL
9. Inserción, Consulta y Borrado de Registros
10. Presentación de Consultas SQL
11. Definiciones de Comunicación

1. Etapa de Análisis

1.1 Recolección de Información

Para comprender el funcionamiento de la clínica y relevar sus necesidades reales, se realizaron entrevistas con el personal médico, administrativo y de recepción. Este proceso permitió identificar las áreas críticas del sistema: la gestión de turnos, el manejo de historias clínicas, el control del inventario y la facturación. Se analizaron los procesos actuales —en su mayoría manuales o dispersos en hojas de cálculo— y se revisó la documentación existente para detectar inconsistencias y oportunidades de mejora.

1.2 Requerimientos del Sistema

A partir de la información recolectada, se identificaron los siguientes requerimientos funcionales que el sistema debe satisfacer:

- **Gestión de Citas:** Programación, modificación y cancelación de turnos médicos.
- **Historias Clínicas Electrónicas:** Creación, consulta y actualización de historias clínicas con acceso seguro.
- **Gestión de Inventarios:** Registro de productos, control de stock y alertas por vencimiento.
- **Facturación y Pagos:** Generación de facturas asociadas a pacientes y registro del estado de pago.
- **Reportes de Gestión:** Generación de reportes estadísticos sobre pacientes, turnos e ingresos.

1.3 Casos de Uso Principales

Se identificaron dos casos de uso centrales que definen las interacciones más importantes entre los actores y el sistema:

Caso de Uso 1 – Programación de Citas

Nombre	Programación de Citas
Actor Principal	Recepcionista / Paciente
Descripción	El sistema permite registrar un turno médico seleccionando paciente, médico, fecha y hora.
Precondición	El paciente debe estar registrado en el sistema y el médico debe tener disponibilidad.
Postcondición	La cita queda registrada en el sistema y disponible para consulta.
Flujo Principal	1. La recepcionista ingresa al módulo de citas. 2. Selecciona el paciente y el médico disponible. 3. Elige la fecha y hora del turno. 4. El sistema guarda el turno con estado "Pendiente".

	5. Se confirma el registro al usuario.
--	--

Caso de Uso 2 – Gestión de Historias Clínicas

Nombre	Gestión de Historias Clínicas
Actor Principal	Médico
Descripción	El médico registra o actualiza la historia clínica de un paciente tras una consulta.
Precondición	El paciente debe tener una cita confirmada con el médico.
Postcondición	La historia clínica queda registrada en la base de datos de forma segura.
Flujo Principal	1. El médico accede al sistema con sus credenciales. 2. Selecciona el paciente a atender. 3. Visualiza la historia clínica existente. 4. Registra el diagnóstico y el tratamiento indicado. 5. El sistema guarda la nueva entrada en la historia clínica.

2. Etapa de Diseño

2.1 Arquitectura del Sistema

El sistema fue diseñado con una arquitectura por capas que separa claramente las responsabilidades de cada componente. La capa de presentación, implementada en Java, interactúa con el usuario a través de un menú de consola. La capa de lógica de negocio procesa las operaciones mediante clases especializadas por entidad. La capa de acceso a datos utiliza el patrón DAO (Data Access Object), que encapsula todas las operaciones con la base de datos y desacopla la lógica de negocio del motor de persistencia. Finalmente, la capa de datos está implementada en MySQL, con tablas normalizadas y relaciones mediante claves foráneas.

2.2 Modelo de Datos Relacional

El modelo de datos fue diseñado para representar todas las entidades del dominio de la clínica y sus relaciones. Se definieron 8 tablas: Pacientes, Médicos, Citas, Historias_Clinicas, Inventarios, Facturas, Proveedores y Reportes. Las relaciones entre ellas se establecen mediante claves foráneas que garantizan la integridad referencial de los datos.

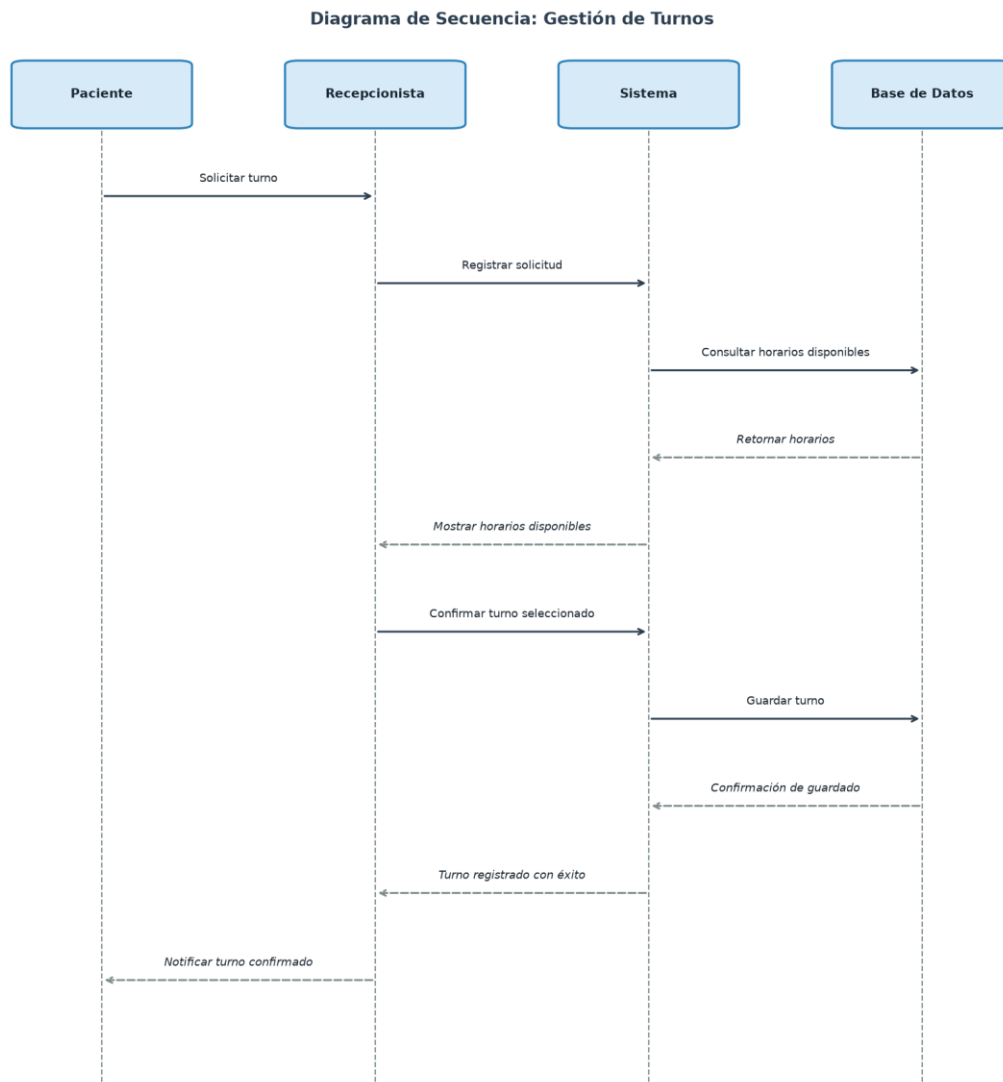
- **Pacientes:** id (PK), nombre, apellido, fecha_nacimiento, dirección, teléfono, correo_electronico
- **Médicos:** id (PK), nombre, apellido, especialidad, teléfono, correo_electronico
- **Citas:** id (PK), paciente_id (FK), medico_id (FK), fecha_hora, estado
- **Historias_Clinicas:** id (PK), paciente_id (FK), medico_id (FK), fecha, detalles
- **Inventarios:** id (PK), producto, cantidad, fecha_vencimiento, proveedor_id (FK)
- **Facturas:** id (PK), paciente_id (FK), fecha, total, estado
- **Proveedores:** id (PK), nombre, contacto
- **Reportes:** id (PK), tipo_reporte, fecha_creacion, detalles

3. Diagramas de Secuencia

Los diagramas de secuencia describen cómo interactúan los actores con el sistema a lo largo del tiempo para completar un proceso específico. Las flechas continuas representan mensajes de invocación y las flechas discontinuas representan respuestas o retornos.

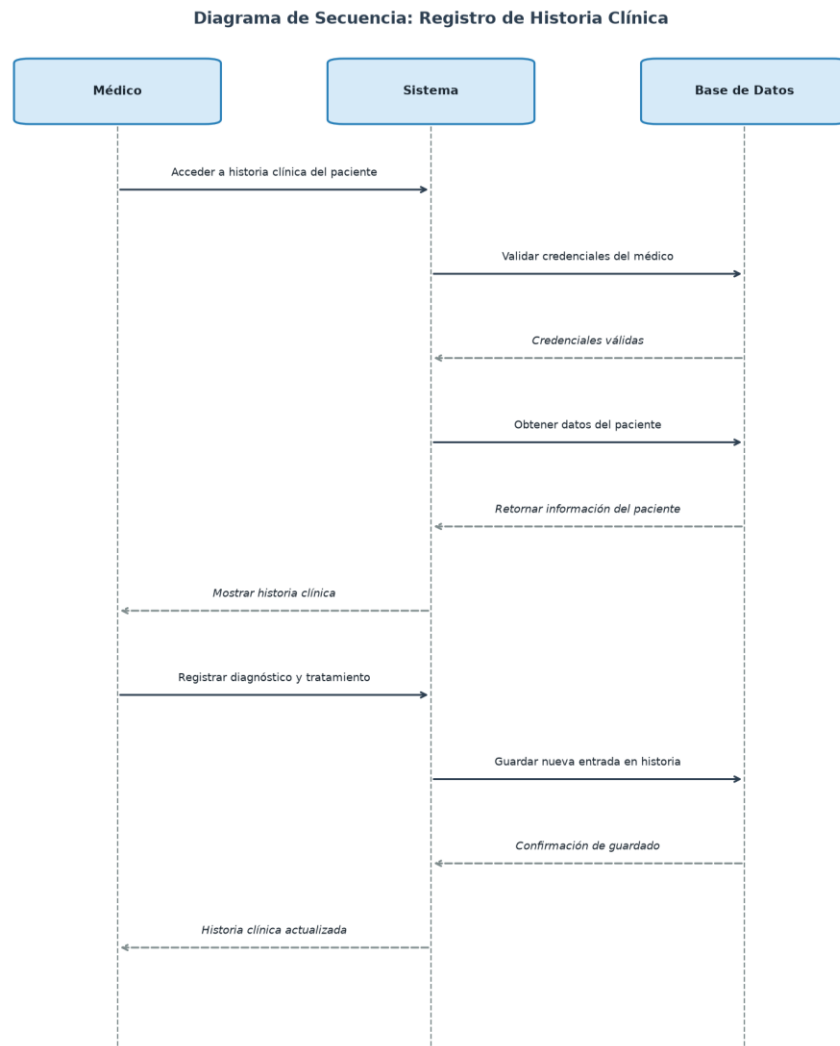
3.1 Gestión de Turnos

El siguiente diagrama muestra el flujo completo para programar un turno médico. El paciente solicita el turno a través de la recepcionista, quien lo registra en el sistema. El sistema consulta los horarios disponibles en la base de datos y, una vez confirmado, guarda el turno y notifica al paciente.



3.2 Registro de Historia Clínica

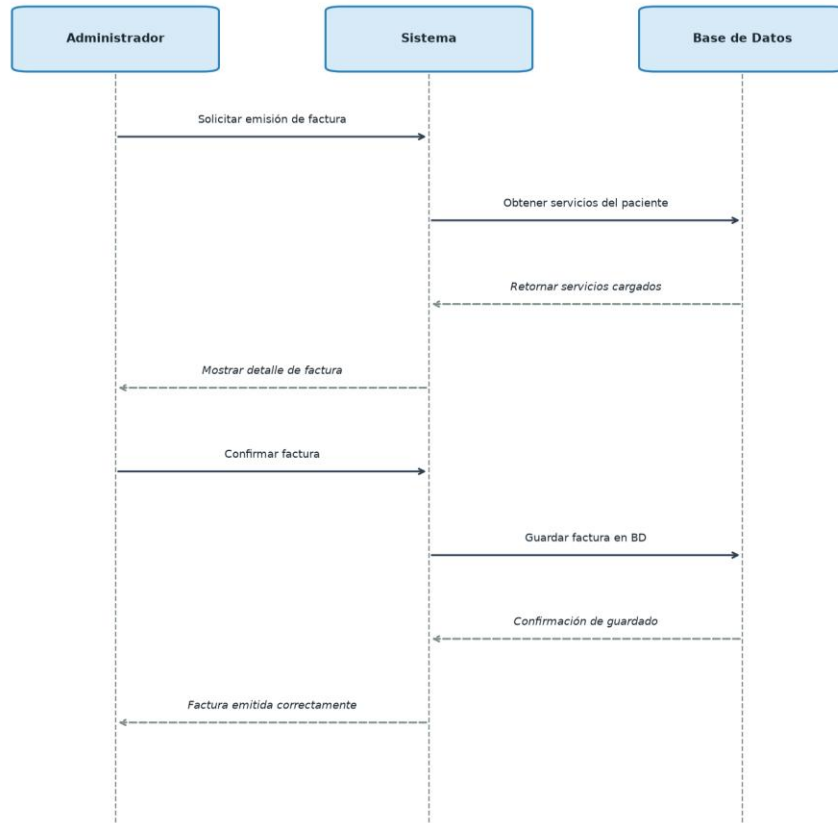
Este diagrama ilustra el proceso mediante el cual un médico accede y actualiza la historia clínica de un paciente. El sistema valida las credenciales del médico, recupera la información existente del paciente y persiste el nuevo diagnóstico o tratamiento en la base de datos.



3.3 Facturación

El diagrama de facturación describe cómo el administrador genera una factura por los servicios prestados. El sistema obtiene los servicios del paciente, muestra el detalle para su confirmación y persiste la factura en la base de datos con su estado correspondiente.

Diagrama de Secuencia: Facturación



4. Etapa de Implementación

4.1 Desarrollo del Sistema

El sistema fue desarrollado íntegramente en Java (JDK 21), organizando el código en módulos independientes por funcionalidad. Se implementó el patrón de diseño DAO (Data Access Object), que permite separar completamente la lógica de negocio del acceso a la base de datos. Esto facilita el mantenimiento del código, ya que cualquier cambio en la capa de datos no impacta en la lógica de la aplicación.

Cada entidad del sistema (Paciente, Médico, Cita, Producto, Factura) cuenta con su propia clase DAO que implementa las operaciones CRUD completas: insertar, obtener por ID, obtener todos, actualizar y eliminar.

4.2 Conexión con la Base de Datos

La conexión a MySQL se realiza a través del driver JDBC (mysql-connector-java). La clase ConexionDB centraliza la configuración de la conexión, evitando duplicación de parámetros en el código:

```
public class ConexionDB {
    private static final String URL =

    "jdbc:mysql://localhost:3306/clinicadb?useSSL=false&serverTimezone=UTC";
    private static final String USER = "root";
    private static final String PASSWORD = "123456";

    public static Connection getConexion() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

Esta clase es utilizada por todos los DAOs a través de AbstractDAO, que establece la conexión en su constructor y la comparte con las subclases.

4.3 Patrón DAO

La interfaz DAO<T> define el contrato que deben cumplir todos los DAOs del sistema. Esto garantiza coherencia en las operaciones disponibles para cada entidad:

```
public interface DAO<T> {
    void insertar(T t) throws SQLException;
    T obtener(int id) throws SQLException;
    List<T> obtenerTodos() throws SQLException;
    void actualizar(T t) throws SQLException;
    void eliminar(int id) throws SQLException;
}
```

Cada DAO concreto (PacienteDAO, MedicoDAO, etc.) extiende AbstractDAO<T> e implementa estas operaciones usando PreparedStatement para prevenir inyección SQL y mejorar el rendimiento en consultas repetidas.

4.4 Repositorio del Proyecto

El código fuente completo del proyecto se encuentra disponible en:

<https://github.com/antonellaanahi/SEMINARIODEPRACTICA>

5. Etapa de Pruebas

Para validar el correcto funcionamiento del sistema se llevaron a cabo distintos niveles de prueba, asegurando tanto la corrección individual de cada componente como la integración entre módulos.

Pruebas Unitarias

Se verificó que cada método de los DAOs funcione correctamente de forma independiente. Por ejemplo, se comprobó que insertar un paciente genera el registro en la base de datos con todos sus atributos correctamente asignados, y que obtener por ID retorna el objeto esperado con los datos persistidos.

Pruebas de Integración

Se validó la interacción entre módulos. En particular, se verificó que al crear una cita los IDs de paciente y médico referenciados existan previamente (clave foránea), y que el módulo de reportes integre correctamente los datos de pacientes, citas e inventarios.

Pruebas de Sistema

Se ejecutaron los flujos completos del sistema: desde el registro de un paciente, pasando por la programación de un turno y el registro de una historia clínica, hasta la emisión de la factura correspondiente. Se verificó que los datos persistan correctamente en cada etapa.

Pruebas de Usuario

Se simularon escenarios de uso real con el menú de consola, incluyendo casos de error como intentar asignar una cita sin pacientes registrados o ingresar un ID inexistente. Se validó que el sistema responda con mensajes claros y no se produzcan excepciones no controladas.

6. Definición de Base de Datos para el Sistema

La base de datos ClinicaDB fue diseñada aplicando las tres primeras formas normales de la normalización relacional, con el objetivo de eliminar redundancias y garantizar la integridad de la información.

Primera Forma Normal (1FN)

Se verificó que cada tabla contenga únicamente atributos atómicos (sin grupos repetitivos ni valores múltiples en un mismo campo). Cada tabla posee una clave primaria única de tipo AUTO_INCREMENT que identifica de forma inequívoca cada registro.

Segunda Forma Normal (2FN)

Todos los atributos de cada tabla dependen funcionalmente de la clave primaria completa. La información de pacientes, médicos, proveedores y facturas se almacena en tablas separadas, evitando que un mismo dato aparezca duplicado en varias tablas. Por ejemplo, los datos del paciente no se repiten en la tabla Citas sino que se referencian mediante la clave foránea paciente_id.

Tercera Forma Normal (3FN)

Se eliminaron las dependencias transitivas. La información de proveedores, por ejemplo, no se almacena dentro de la tabla Inventarios sino en su propia tabla Proveedores, relacionada mediante proveedor_id. De este modo, cualquier cambio en los datos de un proveedor se realiza en un único lugar.

7. Diagrama Entidad-Relación de la Base de Datos

El siguiente diagrama muestra las entidades del sistema, sus atributos y las relaciones entre ellas. Las claves primarias se indican con "PK" y las claves foráneas con "FK". Las relaciones son de tipo uno-a-muchos (1:N) en todos los casos.



Relaciones principales: Pacientes y Médicos tienen relación 1:N con Citas e Historias_Clinicas. Pacientes tiene relación 1:N con Facturas. Proveedores tiene relación 1:N con Inventarios. Reportes es una entidad independiente.

8. Creación de las Tablas MySQL

A continuación se presentan los scripts SQL para la creación de la base de datos y sus tablas. Las claves foráneas aseguran la integridad referencial entre entidades. La tabla Proveedores se crea antes que Inventarios dado que esta última la referencia.

Base de datos

```
CREATE DATABASE IF NOT EXISTS ClinicaDB;
USE ClinicaDB;
```

Tabla Pacientes

```
CREATE TABLE Pacientes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    apellido VARCHAR(50) NOT NULL,
    fecha_nacimiento DATE,
    direccion VARCHAR(100),
    telefono VARCHAR(20),
    correo_electronico VARCHAR(50)
);
```

Tabla Médicos

```
CREATE TABLE Medicos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    apellido VARCHAR(50) NOT NULL,
    especialidad VARCHAR(50),
    telefono VARCHAR(20),
    correo_electronico VARCHAR(50)
);
```

Tabla Proveedores

```
CREATE TABLE Proveedores (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    contacto VARCHAR(100)
);
```

Tabla Citas

```
CREATE TABLE Citas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    paciente_id INT NOT NULL,
    medico_id INT NOT NULL,
    fecha_hora DATETIME NOT NULL,
    estado VARCHAR(20) DEFAULT "Pendiente",
    FOREIGN KEY (paciente_id) REFERENCES Pacientes(id),
    FOREIGN KEY (medico_id) REFERENCES Medicos(id)
);
```

Tabla Historias_Clinicas

```
CREATE TABLE Historias_Clinicas (
    id INT AUTO_INCREMENT PRIMARY KEY,
```



```

    paciente_id INT NOT NULL,
    medico_id INT NOT NULL,
    fecha DATE NOT NULL,
    detalles TEXT,
    FOREIGN KEY (paciente_id) REFERENCES Pacientes(id),
    FOREIGN KEY (medico_id) REFERENCES Medicos(id)
);

```

Tabla Inventarios

```

CREATE TABLE Inventarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    producto VARCHAR(100) NOT NULL,
    cantidad INT DEFAULT 0,
    fecha_vencimiento DATE,
    proveedor_id INT,
    FOREIGN KEY (proveedor_id) REFERENCES Proveedores(id)
);

```

Tabla Facturas

```

CREATE TABLE Facturas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    paciente_id INT NOT NULL,
    fecha DATE NOT NULL,
    total DECIMAL(10,2),
    estado VARCHAR(20) DEFAULT "Pendiente",
    FOREIGN KEY (paciente_id) REFERENCES Pacientes(id)
);

```

Tabla Reportes

```

CREATE TABLE Reportes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    tipo_reporte VARCHAR(50),
    fecha_creacion DATE,
    detalles TEXT
);

```

9. Inserción, Consulta y Borrado de Registros

Las siguientes sentencias SQL muestran las operaciones básicas de manipulación de datos (DML) sobre las tablas principales del sistema.

Inserción de registros

Se insertan registros de ejemplo para pacientes y médicos:

```
INSERT INTO Pacientes (nombre, apellido, fecha_nacimiento, direccion,
telefono, correo_electronico)
VALUES ('Juan', 'Pérez', '1985-03-15', 'Calle Falsa 123', '123456789',
'juan.perez@email.com');
```

```
INSERT INTO Pacientes (nombre, apellido, fecha_nacimiento, direccion,
telefono, correo_electronico)
VALUES ('Maria', 'López', '1990-07-22', 'Av. Siempreviva 742',
'987654321', 'maria.lopez@email.com');
```

```
INSERT INTO Medicos (nombre, apellido, especialidad, telefono,
correo_electronico)
VALUES ('Carlos', 'Gómez', 'Cardiología', '111222333',
'cgomez@clinica.com');
```

```
INSERT INTO Citas (paciente_id, medico_id, fecha_hora, estado)
VALUES (1, 1, '2025-07-10 09:00:00', 'Pendiente');
```

Consulta de registros

Ejemplo de consulta de todos los pacientes registrados:

```
SELECT * FROM Pacientes WHERE apellido = 'Pérez';
```

Borrado de registros

Eliminación de un registro por su clave primaria:

```
DELETE FROM Pacientes WHERE id = 1;
```

10. Presentación de Consultas SQL

A continuación se presentan consultas SQL representativas del sistema, organizadas por funcionalidad:

Listar citas por médico

Obtiene todas las citas asignadas a un médico específico:

```
SELECT c.id, CONCAT(p.nombre, " ", p.apellido) AS paciente,
       c.fecha_hora, c.estado
FROM Citas c
JOIN Pacientes p ON c.paciente_id = p.id
WHERE c.medico_id = 1;
```

Buscar pacientes por nombre

Búsqueda parcial de pacientes usando LIKE:

```
SELECT * FROM Pacientes WHERE nombre LIKE '%Juan%';
```

Inventario próximo a vencer

Productos cuya fecha de vencimiento es en los próximos 30 días:

```
SELECT * FROM Inventarios
WHERE fecha_vencimiento < CURDATE() + INTERVAL 30 DAY
ORDER BY fecha_vencimiento ASC;
```

Total de ingresos por facturas pagadas

Suma de los montos de todas las facturas con estado "Pagada":

```
SELECT SUM(total) AS ingresos_totales
FROM Facturas WHERE estado = 'Pagada';
```

Historial clínico de un paciente

Todas las consultas médicas registradas para un paciente:

```
SELECT h.fecha, CONCAT(m.nombre, " ", m.apellido) AS medico, h.detalles
FROM Historias_Clinicas h
JOIN Medicos m ON h.medico_id = m.id
WHERE h.paciente_id = 1
ORDER BY h.fecha DESC;
```

11. Definiciones de Comunicación

Esta sección define los aspectos relacionados con la comunicación entre los componentes del sistema y con el entorno de red en el que operará.

11.1 Entorno de Red

El sistema utiliza el protocolo TCP/IP para la comunicación entre la aplicación Java y el servidor MySQL. En la configuración actual, tanto la aplicación como la base de datos se ejecutan en el mismo equipo (localhost, puerto 3306), lo que elimina latencia de red en el entorno de desarrollo. En un entorno productivo, la base de datos podría ubicarse en un servidor dedicado, siendo necesario configurar reglas de firewall y acceso remoto en MySQL para permitir la conexión.

11.2 Seguridad en la Transmisión

La conexión JDBC se configura con el parámetro useSSL=false para el entorno de desarrollo local. En un entorno de producción, se recomienda habilitar SSL/TLS para cifrar la transmisión de datos entre la aplicación y el servidor MySQL, protegiendo información sensible como datos de pacientes y registros médicos.

11.3 Integridad y Confidencialidad

La integridad de los datos se garantiza mediante el uso de transacciones y restricciones de clave foránea a nivel de base de datos. El acceso a la información está controlado por el sistema de autenticación de MySQL (usuario y contraseña). Para un entorno productivo, se recomienda implementar roles de usuario en la base de datos, limitando los permisos de cada perfil (lectura, escritura, administración) según su función.

11.4 Interoperabilidad

El sistema fue desarrollado con Java, un lenguaje multiplataforma, lo que permite su ejecución en distintos sistemas operativos sin modificaciones. La base de datos MySQL es ampliamente compatible con otros sistemas y herramientas de análisis. En futuras versiones, se podría exponer una API RESTful para facilitar la integración con sistemas externos, como laboratorios, obras sociales o plataformas de teleasistencia.

TRABAJO PRÁCTICO N° 3

Seminario de Práctica de Informática

Sistema de Gestión Integral para una Clínica de Salud

Alumna: Antonella Diaz

DNI: 28.910.424

Comisión: VINFOR12606

Índice

1. Explicación del Desarrollo en Java
 - 1.1 Estructura general del sistema
 - 1.2 Conceptos de POO aplicados
 - 1.3 Diagrama de Clases UML
2. Presentación del Desarrollo en Java
 - 2.1 Main.java
 - 2.2 SistemaGestionClinica.java
 - 2.3 Persona.java (clase abstracta)
 - 2.4 Paciente.java
 - 2.5 Medico.java
 - 2.6 Cita.java
 - 2.7 Producto.java e Inventario.java
 - 2.8 Factura.java y FacturaDAO.java
 - 2.9 Patrón DAO y ConexionDB
3. Compilación y Ejecución del Programa

1. Explicación del Desarrollo en Java

1.1 Estructura General del Sistema

El sistema fue desarrollado aplicando los principios de la Programación Orientada a Objetos (POO) con el objetivo de construir un software modular, mantenible y extensible. La decisión de separar cada entidad del dominio —pacientes, médicos, citas, inventarios y facturas— en clases propias responde a la necesidad de que cada componente tenga una única responsabilidad bien definida, facilitando tanto su comprensión como su modificación futura sin afectar al resto del sistema.

La arquitectura final se organiza en tres capas: la capa de presentación (menú de consola en SistemaGestionClinica), la capa de lógica de negocio (clases de dominio como Paciente, Medico, Cita, Producto y Factura) y la capa de acceso a datos (patrón DAO con conexión a MySQL mediante JDBC). Esta separación permite, por ejemplo, reemplazar la base de datos sin necesidad de modificar la lógica de negocio.

1.2 Conceptos de POO Aplicados

Encapsulamiento

Todos los atributos de las clases de dominio se declararon como privados (private). El acceso y la modificación de estos datos se realizan exclusivamente a través de métodos públicos (getters y setters). Esta decisión protege la integridad del objeto: por ejemplo, no es posible asignar directamente un valor inválido al campo "estado" de una Cita sin pasar por el método correspondiente, donde se podría agregar validación en el futuro.

Herencia

Se definió la clase abstracta Persona como base para Paciente y Medico. Esta decisión evita duplicar atributos comunes (nombre, apellido, dirección, teléfono, correo) en ambas clases. Al heredar de Persona, tanto Paciente como Medico incorporan automáticamente estos atributos y sus métodos de acceso, y sólo agregan lo que les es propio: la fecha de nacimiento en Paciente y la especialidad en Medico. Esto reduce significativamente el código repetido y mantiene una jerarquía clara.

Polimorfismo

El método toString() fue sobrescrito (override) en cada clase para devolver una representación legible del objeto. Esto permite mostrar información de cualquier entidad de forma uniforme, independientemente de si es un Paciente o un Medico. El mismo mecanismo se aplica en el patrón DAO: la interfaz DAO<T> define operaciones genéricas que cada DAO concreto implementa según las particularidades de su tabla. El sistema puede llamar a insertar() o obtenerTodos() sin conocer si está operando sobre pacientes, médicos o facturas.

Abstracción

Se utilizó la abstracción en dos niveles: la clase abstracta Persona, que define la estructura común pero delega los detalles a sus subclasses, y la interfaz DAO<T>, que establece el

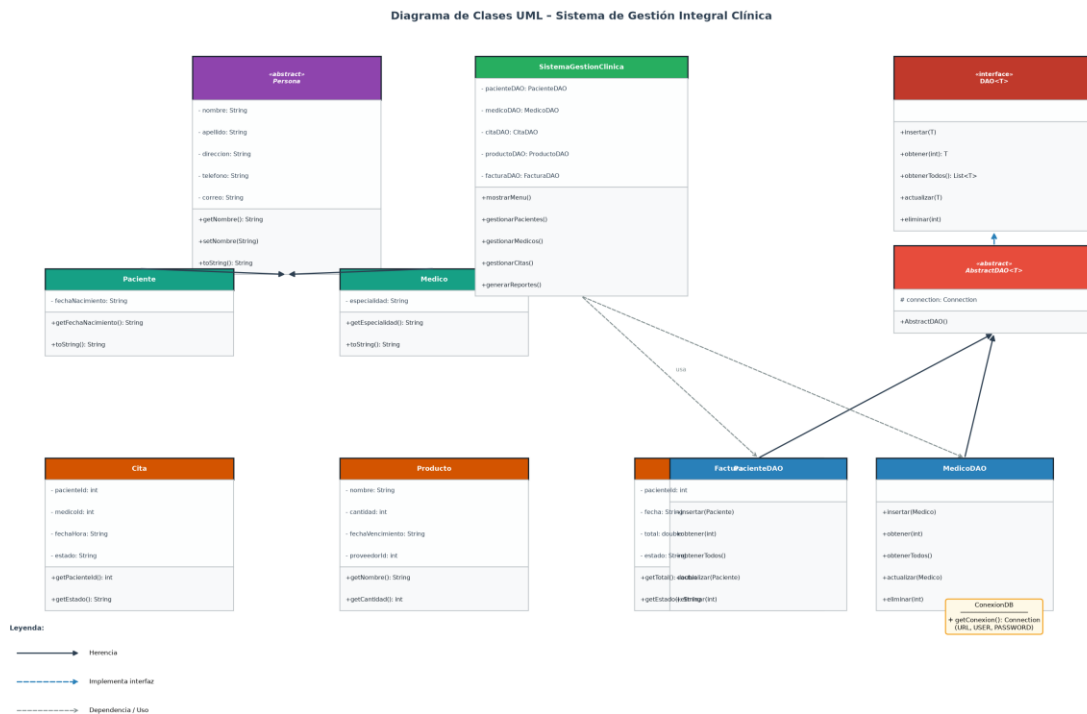
contrato de las operaciones de persistencia sin definir su implementación. Esta capa de abstracción permite que el resto del sistema dependa de contratos (interfaces) y no de implementaciones concretas, un principio clave del diseño orientado a objetos conocido como "programar hacia la interfaz, no hacia la implementación".

Manejo de Excepciones

Las operaciones de base de datos pueden fallar por múltiples razones: conexión perdida, violación de clave foránea, duplicados, etc. Para manejar estos escenarios sin que el programa se detenga abruptamente, se utilizó el manejo de excepciones con bloques try-catch en todos los métodos de los DAOs. Las excepciones de tipo SQLException se capturan y se muestra un mensaje descriptivo al usuario, permitiendo que el sistema continúe operando ante un error puntual.

1.3 Diagrama de Clases UML

El siguiente diagrama muestra la estructura de clases del sistema, sus atributos, métodos y las relaciones entre ellas. Las flechas con triángulo vacío representan herencia, las flechas punteadas con triángulo indican implementación de interfaz, y las flechas punteadas simples representan relaciones de uso o dependencia.



2. Presentación del Desarrollo en Java

2.1 Main.java

Esta clase contiene el método `main()`, que es el punto de entrada del programa. Su función es instanciar el sistema de gestión y delegar el control al menú principal. Se decidió mantener esta clase lo más simple posible, siguiendo el principio de responsabilidad única: Main sólo inicia la aplicación, sin contener lógica de negocio.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        SistemaGestionClinica sistema = new SistemaGestionClinica();
        sistema.mostrarMenu();
    }
}
```

2.2 SistemaGestionClinica.java

Es la clase principal de la aplicación. Centraliza el menú de opciones y coordina el acceso a cada módulo del sistema. En lugar de implementar la lógica directamente, delega las operaciones a los DAOs correspondientes, actuando como un controlador que conecta la interfaz de usuario con la capa de datos. Inicializa una instancia de cada DAO al arrancar, lo que abre la conexión con la base de datos de forma anticipada y la mantiene disponible durante toda la sesión.

```
public class SistemaGestionClinica {
    private PacienteDAO pacienteDAO = new PacienteDAO();
    private MedicoDAO medicoDAO = new MedicoDAO();
    private CitaDAO citaDAO = new CitaDAO();
    private ProductoDAO productoDAO = new ProductoDAO();
    private FacturaDAO facturaDAO = new FacturaDAO();
    private Inventario inventario = new Inventario();

    public void mostrarMenu() {
        Scanner scanner = new Scanner(System.in);
        int opcion;
        do {
            System.out.println("=== Sistema de Gestion Integral ===");
            System.out.println("1. Gestion de Pacientes");
            System.out.println("2. Gestion de Medicos");
            System.out.println("3. Gestion de Citas");
            System.out.println("4. Historias Clinicas");
            System.out.println("5. Gestion de Inventarios");
            System.out.println("6. Facturacion y Pagos");
            System.out.println("7. Reportes de Gestion");
            System.out.println("8. Salir");
            System.out.print("Seleccione una opcion: ");
            opcion = scanner.nextInt();
            // ... switch con llamadas a métodos privados por módulo
        } while (opcion != 8);
    }
}
```

2.3 Persona.java (Clase Abstracta)

Persona es la clase base de la jerarquía de herencia. Se definió como abstracta porque no tiene sentido instanciar una "Persona" genérica en el contexto de la clínica: siempre se trabaja con Pacientes o Médicos específicos. Sin embargo, ambos comparten los mismos atributos de identificación personal, por lo que centralizar estos datos en Persona elimina duplicación y facilita el mantenimiento: si en el futuro se agrega un atributo (como número de documento), se modifica en un único lugar y ambas subclases lo heredan automáticamente.

```
public abstract class Persona {
    private int    id;
    private String nombre;
    private String apellido;
    private String direccion;
    private String telefono;
    private String correo;

    // Constructor, getters y setters
    public String getNombre() { return nombre; }
    public String getApellido(){ return apellido; }
    // ... resto de métodos de acceso
}
```

2.4 Paciente.java

Paciente extiende Persona y agrega el atributo fechaNacimiento, que es específico de los pacientes y no aplica a los médicos. Gracias a la herencia, un objeto Paciente tiene disponibles todos los métodos de Persona sin necesidad de redefinirlos. El constructor llama a `super()` para inicializar los atributos heredados y luego asigna el propio.

```
public class Paciente extends Persona {
    private String fechaNacimiento;

    public Paciente(String nombre, String apellido, String fechaNacimiento,
                    String direccion, String telefono, String correo) {
        super();
        setNombre(nombre); setApellido(apellido);
        setDireccion(direccion); setTelefono(telefono); setCorreo(correo);
        this.fechaNacimiento = fechaNacimiento;
    }
    public String getFechaNacimiento() { return fechaNacimiento; }
}
```

2.5 Medico.java

De forma análoga a Paciente, Medico hereda de Persona y agrega el atributo especialidad. Esta diferenciación permite que el sistema trate a médicos y pacientes de forma polimórfica cuando sea necesario (por ejemplo, para mostrar listas combinadas) y de forma específica cuando se requieren sus atributos propios.

```
public class Medico extends Persona {
    private String especialidad;

    public Medico(String nombre, String apellido, String especialidad,
```

```

        String telefono, String correo) {
    super();
    setNombre(nombre); setApellido(apellido);
    setTelefono(telefono); setCorreo(correo);
    this.especialidad = especialidad;
}
public String getEspecialidad() { return especialidad; }
}

```

2.6 Cita.java

Cita representa un turno médico y relaciona a un paciente con un médico en una fecha y hora determinada. En lugar de almacenar los objetos completos de Paciente y Medico, se optó por guardar únicamente sus IDs (claves foráneas), siguiendo el mismo modelo que la base de datos. Esto reduce el uso de memoria y evita inconsistencias entre el objeto en memoria y los datos persistidos.

```

public class Cita {
    private int id;
    private int pacienteId;
    private int medicoId;
    private String fechaHora;
    private String estado;

    public Cita(int pacienteId, int medicoId, String fechaHora, String estado)
    {
        this.pacienteId = pacienteId;
        this.medicoId = medicoId;
        this.fechaHora = fechaHora;
        this.estado = estado;
    }
    // Getters y setters
}

```

2.7 Producto.java e Inventario.java

Producto almacena la información de cada ítem del inventario: nombre, cantidad, fecha de vencimiento y proveedor. La clase Inventario actúa como contenedor de productos y ofrece operaciones de alto nivel como agregar, eliminar, buscar y mostrar. Esta separación respeta el principio de responsabilidad única: Producto sabe qué es un producto, e Inventario sabe cómo gestionar una colección de ellos. Adicionalmente, la clase EstadisticasInventario calcula el promedio de unidades disponibles usando un array de Producto[], demostrando el uso de arrays en el contexto real.

```

public class Inventario {
    private ArrayList<Producto> productos = new ArrayList<>();

    public void agregarProducto(Producto p) { productos.add(p); }
    public void eliminarProducto(Producto p) { productos.remove(p); }
    public Producto buscarProducto(String nombre) {
        for (Producto p : productos)
            if (p.getNombre().equalsIgnoreCase(nombre)) return p;
        return null;
    }
    public void mostrarInventario() {

```

```

        for (Producto p : productos)
            System.out.printf("%d | %s | %d | %s\n",
                               p.getId(), p.getNombre(), p.getCantidad(),
                               p.getFechaVencimiento());
    }
}

```

2.8 Factura.java

Factura representa un comprobante de pago vinculado a un paciente. El atributo estado permite distinguir entre facturas pendientes de cobro y facturas ya pagadas, lo que resulta esencial para el módulo de reportes, que calcula el total de ingresos sumando únicamente las facturas con estado "Pagada". El total se define como double para soportar valores con decimales (centavos).

```

public class Factura {
    private int id;
    private int pacienteId;
    private String fecha;
    private double total;
    private String estado;

    public Factura(int pacienteId, String fecha, double total, String estado) {
        this.pacienteId = pacienteId;
        this.fecha = fecha;
        this.total = total;
        this.estado = estado;
    }

    public double getTotal() { return total; }
    public String getEstado() { return estado; }
}

```

2.9 Patrón DAO y ConexionDB

El patrón DAO (Data Access Object) fue elegido como estrategia de acceso a datos porque permite separar completamente la lógica de negocio de las operaciones SQL. La interfaz DAO<T> define el contrato genérico con cinco operaciones CRUD. La clase abstracta AbstractDAO<T> implementa esta interfaz y establece la conexión a la base de datos en su constructor, compartiéndola con todas las subclases. Cada DAO concreto (PacienteDAO, MedicoDAO, CitaDAO, ProductoDAO, FacturaDAO) hereda de AbstractDAO y provee la implementación SQL específica para su tabla.

```

public interface DAO<T> {
    void insertar(T t) throws SQLException;
    T obtener(int id) throws SQLException;
    List<T> obtenerTodos() throws SQLException;
    void actualizar(T t) throws SQLException;
    void eliminar(int id) throws SQLException;
}

public abstract class AbstractDAO<T> implements DAO<T> {
    protected Connection connection;
    public AbstractDAO() {
        try {
            this.connection = ConexionDB.getConexion();
        }
    }
}

```

```

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

La clase `ConexionDB` centraliza los parámetros de conexión (URL, usuario, contraseña) en un único lugar. Si en el futuro el sistema migra a otro servidor o motor de base de datos, sólo se modifica esta clase, sin tocar ningún DAO:

```

public class ConexionDB {
    private static final String URL =

    "jdbc:mysql://localhost:3306/clinicadb?useSSL=false&serverTimezone=UTC";
    private static final String USER    = "root";
    private static final String PASSWORD = "123456";

    public static Connection getConexion() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}

```

Ejemplo de DAO concreto — `PacienteDAO.insertar()` usa un `PreparedStatement` para evitar inyección SQL y asigna los parámetros con sus tipos correctos:

```

public void insertar(Paciente paciente) {
    String query = "INSERT INTO pacientes "
        + "(nombre, apellido, fecha_nacimiento, direccion, telefono, "
        + "correo) "
        + "VALUES (?, ?, ?, ?, ?, ?)";
    try (PreparedStatement ps = connection.prepareStatement(query)) {
        ps.setString(1, paciente.getNombre());
        ps.setString(2, paciente.getApellido());
        ps.setString(3, paciente.getFechaNacimiento());
        ps.setString(4, paciente.getDireccion());
        ps.setString(5, paciente.getTelefono());
        ps.setString(6, paciente.getCorreo());
        ps.executeUpdate();
    } catch (SQLException e) {
        System.err.println("Error al insertar paciente: " + e.getMessage());
    }
}

```

3. Compilación y Ejecución del Programa

Para compilar y ejecutar el sistema es necesario contar con el JDK instalado y el driver JDBC de MySQL (mysql-connector.jar) en la misma carpeta que los archivos .java. El sistema fue probado con JDK 21 y MySQL Connector/J 9.1.0.

Requisitos previos

- JDK 21 o superior instalado
- MySQL Server en ejecución con la base de datos ClinicaDB creada
- Archivo mysql-connector.jar en la misma carpeta que los .java

Compilación

Desde la terminal, en la carpeta que contiene los archivos Java:

```
javac -cp ".;mysql-connector.jar" *.java
# En Linux/Mac usar ":" en lugar de ";":
javac -cp ".:mysql-connector.jar" *.java
```

El flag -cp especifica el classpath: incluye el directorio actual (.) y el driver JDBC. Compilar todos los archivos juntos (*.java) asegura que las dependencias entre clases se resuelvan correctamente.

Ejecución

```
java -cp ".;mysql-connector.jar" Main
```

Al ejecutarse, el sistema establece la conexión con MySQL y presenta el menú principal. Desde allí el usuario puede gestionar pacientes, médicos, citas, historias clínicas, inventarios, facturación y generar reportes de gestión, con todos los datos persistidos en la base de datos ClinicaDB.

Resultado esperado

```
=== Sistema de Gestion Integral para Clinicas de Salud ===
1. Gestion de Pacientes
2. Gestion de Medicos
3. Gestion de Citas
4. Historias Clinicas
5. Gestion de Inventarios
6. Facturacion y Pagos
7. Reportes de Gestion
8. Salir
Seleccione una opcion: _
```

Cada opción despliega un submenú que permite ver los registros existentes o ingresar nuevos datos. Los reportes muestran estadísticas en tiempo real: total de pacientes, médicos, citas pendientes, promedio de stock en inventario e ingresos totales por facturas pagadas.

TRABAJO PRÁCTICO NRO 4
SEMINARIO DE PRÁCTICA
DE INFORMATICA

Sistema de Gestión Integral
para una Clínica de Salud

Antonella Diaz
DNI 28.910.424
VINFOR12606

<https://github.com/antonellaanahi/SEMINARIODEPRACTICA>

Índice

1. Selección del patrón de diseño y justificación.
2. Persistencia y consulta de datos en una base de datos MySQL. Esta actividad incluye establecer conexiones, realizar consultas, actualizar registros y presentar resultados en la interfaz.
3. Correcta aplicación de excepciones para la interacción con la base de datos MySQL.
4. Inclusión pertinente de clases abstractas o interfaces.
5. Utilización complementaria de arreglos y de la clase ArrayList

Consideración:

La Clínica de Salud requiere un sistema que permita administrar de manera centralizada la información de pacientes, médicos, turnos, inventario y facturación. Actualmente estos procesos pueden realizarse mediante registros manuales o utilizando diferentes aplicaciones independientes, lo que genera demoras, duplicación de información y una mayor posibilidad de cometer errores.

El proyecto propone desarrollar un Sistema de Gestión Integral que reúna toda la información en una única plataforma. De esta manera se optimizan las tareas administrativas, se mejora la organización de los datos y se brinda una atención más eficiente a los pacientes.

Los principales usuarios del sistema serán el personal de recepción, los médicos, el área administrativa y los responsables del control del inventario.

Proceso Unificado de Desarrollo:

El desarrollo del proyecto se realizó siguiendo las etapas del Proceso Unificado de Desarrollo (PUD), permitiendo organizar el trabajo de forma iterativa e incremental.

Inicio: se identificó la problemática de la clínica y se definieron los objetivos del sistema.

Elaboración: se analizaron los requerimientos, se diseñó la arquitectura y se seleccionó el patrón DAO para el acceso a los datos.

Construcción: se desarrolló el prototipo utilizando Java y una base de datos MySQL, implementando la persistencia de la información y el manejo de excepciones.

Transición: se realizaron pruebas de funcionamiento para verificar las operaciones de alta, baja, modificación y consulta de los datos.

Desarrollo:

Para continuar con el desarrollo del Sistema de Gestión Integral para Clínicas de Salud, vamos a incluir una selección de patrón de diseño, persistencia y consulta de datos en una base de datos MySQL, correcta aplicación de excepciones para la interacción con la base de datos, inclusión de clases abstractas o interfaces y utilización de arreglos y la clase ArrayList.

1. Selección del Patrón de Diseño y Justificación

Patrón de Diseño: DAO (Data Access Object)

Justificación: Se seleccionó el patrón DAO (Data Access Object) porque permite separar la lógica de acceso a los datos de la lógica de negocio del sistema. Gracias a esta arquitectura, cualquier modificación en la base de datos puede realizarse sin afectar el funcionamiento del resto de la aplicación. Además, facilita el mantenimiento del código, mejora la reutilización de componentes y permite escalar el sistema de manera más sencilla a medida que se incorporen nuevos módulos.

2. Persistencia y Consulta de Datos en una Base de Datos MySQL

Configuración del Entorno

Primero, necesitamos agregar la biblioteca de MySQL Connector a nuestro proyecto. Esto se puede hacer descargando el archivo JAR del conector MySQL y añadiéndolo al classpath del proyecto.

Dependencia Maven :

En XML

```
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.27</version>
</dependency>
```

Configuración de la Base de Datos MySQL

Creamos una base de datos MySQL llamada clinica y las tablas necesarias para almacenar la información de pacientes, médicos, citas, inventarios y facturación.

En SQL

Para almacenar la información del sistema se diseñó una base de datos relacional denominada ClinicaDB, compuesta por 8 tablas que representan las principales entidades del negocio. Se incorporaron tablas adicionales como Historias_Clinicas, Proveedores y Reportes para cubrir todas las funcionalidades del sistema. La estructura mantiene la integridad de los datos mediante claves primarias y claves foráneas.

```
CREATE DATABASE IF NOT EXISTS ClinicaDB;
USE ClinicaDB;

CREATE TABLE Pacientes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50),
  apellido VARCHAR(50),
  fecha_nacimiento DATE,
  direccion VARCHAR(100),
  telefono VARCHAR(20),
  correo_electronico VARCHAR(50)
);

CREATE TABLE Medicos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50),
  apellido VARCHAR(50),
  especialidad VARCHAR(50),
  telefono VARCHAR(20),
  correo_electronico VARCHAR(50)
);

CREATE TABLE Proveedores (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100),
  contacto VARCHAR(100)
);

CREATE TABLE Citas (
  id INT AUTO_INCREMENT PRIMARY KEY,
  paciente_id INT,
  medico_id INT,
  fecha_hora DATETIME,
  estado VARCHAR(20),
  FOREIGN KEY (paciente_id) REFERENCES Pacientes(id),
  FOREIGN KEY (medico_id) REFERENCES Medicos(id)
);

CREATE TABLE Historias_Clinicas (
  id INT AUTO_INCREMENT PRIMARY KEY,
  paciente_id INT,
  medico_id INT,
```

```

    fecha DATE,
    detalles TEXT,
    FOREIGN KEY (paciente_id) REFERENCES Pacientes(id),
    FOREIGN KEY (medico_id) REFERENCES Medicos(id)
);

CREATE TABLE Inventarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    producto VARCHAR(100),
    cantidad INT,
    fecha_vencimiento DATE,
    proveedor_id INT,
    FOREIGN KEY (proveedor_id) REFERENCES Proveedores(id)
);

CREATE TABLE Facturas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    paciente_id INT,
    fecha DATE,
    total DECIMAL(10, 2),
    estado VARCHAR(20),
    FOREIGN KEY (paciente_id) REFERENCES Pacientes(id)
);

CREATE TABLE Reportes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    tipo_reporte VARCHAR(50),
    fecha_creacion DATE,
    detalles TEXT
);

```

Código Java para Conexión y Manejo de Base de Datos

Clase de Conexión a la Base de Datos:

La conexión entre la aplicación desarrollada en Java y la base de datos MySQL se realiza mediante JDBC. Para ello se implementó la clase ConexionDB que establece la conexión utilizando el driver MySQL Connector/J 9.1.0, descargado directamente desde Maven Central y añadido al classpath del proyecto.

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

```

```

public class ConexionDB {
    private static final String URL =
        "jdbc:mysql://localhost:3306/clinicadb?useSSL=false&serverTimezone=UTC";
    private static final String USER = "root";
    private static final String PASSWORD = "123456";

    public static Connection getConexion() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}

```

DAO Interface y Clase Abstracta:

Se implementó la clase SistemaGestionClinica que centraliza toda la lógica del sistema a través de un menú interactivo con 8 opciones: gestión de pacientes, médicos, citas, historias clínicas, inventarios, facturación, reportes y salida. Cada operación obtiene una conexión mediante ConexionDB y utiliza PreparedStatement para ejecutar las consultas de forma segura, manejando los errores con bloques try-catch que capturan SQLException.

```

import java.util.List;

public interface DAO<T> {
    void insertar(T t) throws SQLException;
    T obtener(int id) throws SQLException;
    List<T> obtenerTodos() throws SQLException;
    void actualizar(T t) throws SQLException;
    void eliminar(int id) throws SQLException;
}

public abstract class AbstractDAO<T> implements DAO<T> {
    protected Connection connection;

    public AbstractDAO() {
        try {
            this.connection = DatabaseConnection.getConnection();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

DAO de Paciente:

La clase PacienteDAO implementa los métodos definidos en la interfaz para administrar la información de los pacientes. En ella se desarrollan las operaciones de inserción, consulta, modificación y eliminación utilizando consultas SQL preparadas (PreparedStatement), lo que además mejora la seguridad frente a ataques de inyección SQL.

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class PacienteDAO extends AbstractDAO<Paciente> {
    @Override
    public void insertar(Paciente paciente) throws SQLException {
        String query = "INSERT INTO Paciente (nombre, direccion, telefono,
numeroHistoriaClinica) VALUES (?, ?, ?, ?)";
        PreparedStatement ps = connection.prepareStatement(query);
        ps.setString(1, paciente.getNombre());
        ps.setString(2, paciente.getDireccion());
        ps.setString(3, paciente.getTelefono());
        ps.setString(4, paciente.getNumeroHistoriaClinica());
        ps.executeUpdate();
    }

    @Override
    public Paciente obtener(int id) throws SQLException {
        String query = "SELECT * FROM Paciente WHERE id = ?";
        PreparedStatement ps = connection.prepareStatement(query);
        ps.setInt(1, id);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            return new Paciente(
                rs.getString("nombre"),
                rs.getString("direccion"),
                rs.getString("telefono"),
                rs.getString("numeroHistoriaClinica")
            );
        }
        return null;
    }

    @Override
    public List<Paciente> obtenerTodos() throws SQLException {
        String query = "SELECT * FROM Paciente";
        PreparedStatement ps = connection.prepareStatement(query);
        ResultSet rs = ps.executeQuery();
        List<Paciente> pacientes = new ArrayList<>();
        while (rs.next()) {
            pacientes.add(new Paciente(
                rs.getString("nombre"),
                rs.getString("direccion"),
                rs.getString("telefono"),
                rs.getString("numeroHistoriaClinica")
            ));
        }
    }
}
```

```

    ));
}
return pacientes;
}

@Override
public void actualizar(Paciente paciente) throws SQLException {
    String query = "UPDATE Paciente SET nombre = ?, direccion = ?, telefono = ?,
numeroHistoriaClinica = ? WHERE id = ?";
    PreparedStatement ps = connection.prepareStatement(query);
    ps.setString(1, paciente.getNombre());
    ps.setString(2, paciente.getDireccion());
    ps.setString(3, paciente.getTelefono());
    ps.setString(4, paciente.getNumeroHistoriaClinica());
    ps.setInt(5, paciente.getId());
    ps.executeUpdate();
}

@Override
public void eliminar(int id) throws SQLException {
    String query = "DELETE FROM Paciente WHERE id = ?";
    PreparedStatement ps = connection.prepareStatement(query);
    ps.setInt(1, id);
    ps.executeUpdate();
}
}

```

3. Correcta Aplicación de Excepciones

Al interactuar con la base de datos, es crucial manejar adecuadamente las excepciones para asegurar la robustez del sistema. Utilizaremos bloques try-catch para manejar SQLException y otras posibles excepciones.

La utilización de bloques try-catch permite detectar errores durante la interacción con la base de datos sin interrumpir la ejecución de la aplicación. Esto facilita informar al usuario cuando ocurre un problema y simplifica las tareas de mantenimiento y depuración del sistema.

Ejemplo de Manejo de Excepciones en PacienteDAO:

```

public class PacienteDAO extends AbstractDAO<Paciente> {
    @Override
    public void insertar(Paciente paciente) {
        String query = "INSERT INTO Paciente (nombre, direccion, telefono,
numeroHistoriaClinica) VALUES (?, ?, ?, ?)";
        try (PreparedStatement ps = connection.prepareStatement(query)) {
            ps.setString(1, paciente.getNombre());
            ps.setString(2, paciente.getDireccion());
            ps.setString(3, paciente.getTelefono());
            ps.setString(4, paciente.getNumeroHistoriaClinica());
            ps.executeUpdate();
        }
    }
}

```

```

    } catch (SQLException e) {
        System.err.println("Error al insertar paciente: " + e.getMessage());
    }
}

@Override
public Paciente obtener(int id) {
    String query = "SELECT * FROM Paciente WHERE id = ?";
    try (PreparedStatement ps = connection.prepareStatement(query)) {
        ps.setInt(1, id);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            return new Paciente(
                rs.getString("nombre"),
                rs.getString("direccion"),
                rs.getString("telefono"),
                rs.getString("numeroHistoriaClinica")
            );
        }
    } catch (SQLException e) {
        System.err.println("Error al obtener paciente: " + e.getMessage());
    }
    return null;
}

// Implementación de otros métodos con manejo de excepciones similar...
}

```

4. Inclusión de Clases Abstractas o Interfaces

En el desarrollo del sistema se implementó una interfaz denominada DAO<>, que define las operaciones básicas de acceso a los datos (insertar, obtener, actualizar y eliminar). De esta manera, todas las clases encargadas de interactuar con la base de datos mantienen una estructura común y uniforme.

Asimismo, se creó una clase abstracta denominada AbstractDAO<>, cuya función es centralizar la conexión con la base de datos MySQL. Gracias a esta implementación, las clases específicas como PacienteDAO heredan la conexión y evitan duplicar código, favoreciendo la reutilización y el mantenimiento de la aplicación.

La utilización conjunta de interfaces y clases abstractas permite aplicar los principios de la Programación Orientada a Objetos, especialmente la abstracción, la reutilización del código y el polimorfismo. Además, facilita la incorporación de nuevas entidades al sistema, como MedicoDAO o FacturaDAO, manteniendo la misma estructura de desarrollo y respetando el patrón DAO seleccionado.

5. Utilización Complementaria de Arreglos y de la Clase ArrayList

Para administrar colecciones dinámicas de objetos se utilizó la clase ArrayList, ya que permite agregar y eliminar elementos durante la ejecución del programa. En cambio, los arreglos se emplean cuando la cantidad de elementos es conocida y permanece constante, como sucede en el cálculo de determinadas estadísticas.

Usaremos ArrayList para manejar listas de objetos dinámicamente, mientras que los arreglos pueden ser utilizados para operaciones específicas donde el tamaño es fijo.

Ejemplo de uso de ArrayList en Inventario:

```
public class Inventario {
    private ArrayList<Producto> productos;

    public Inventario() {
        productos = new ArrayList<>();
    }

    public void agregarProducto(Producto producto) {
        productos.add(producto);
    }

    public void eliminarProducto(Producto producto) {
        productos.remove(producto);
    }

    public Producto buscarProducto(String nombre) {
        for (Producto producto : productos) {
            if (producto.getNombre().equalsIgnoreCase(nombre)) {
                return producto;
            }
        }
        return null;
    }

    public void mostrarInventario() {
        for (Producto producto : productos) {
            System.out.println(producto);
        }
    }
}
```

Ejemplo de uso de Arreglos:

Uso de Arreglo para una Operación Específica:

```
public class EstadisticasInventario {
    public static double calcularPromedioCantidad(Producto[] productos) {
        int total = 0;
        for (Producto producto : productos) {
```

```
        total += producto.getCantidad();
    }
    return total / (double) productos.length;
}
}
```

Conclusión:

El desarrollo del Sistema de Gestión Integral para Clínicas de Salud permitió aplicar de forma práctica los contenidos trabajados durante el módulo. La implementación del patrón DAO mejoró la organización del código al separar la lógica de acceso a los datos de la lógica de negocio, mientras que la utilización de MySQL garantizó una adecuada persistencia de la información.

Asimismo, la incorporación de interfaces, clases abstractas, manejo de excepciones, arreglos y la clase ArrayList permitió desarrollar un prototipo más organizado, reutilizable y escalable. Como mejora futura, el sistema podría incorporar funcionalidades como la gestión de historias clínicas digitales, turnos en línea y generación de reportes estadísticos para optimizar aún más la administración de la clínica.